

Order Processing System – OOP Task

مقدمة بسيطة

الهدف منه إن الطالب يطبق كل (Order Processing System) التاسك ده عبارة عن نظام بسيط لإدارة ومعالجة الطلبات بشكل عملي في مشروع شبه واقعي (Object-Oriented Programming) OOP مفاهيم الـ

المطلوب من المشروع

(تصميم نظام لإدارة الطلبات داخل شركة (زي متجر أونلاين بسيط

النظام لازم يقدر يعمل الآتي:

- إنشاء عميل (Customer)
- إنشاء منتجات (Products)
- إنشاء طلب (Order)
- إضافة منتجات للطلب
- حساب إجمالي الطلب
- تطبيق خصومات أو رسوم (اختياري)
- متابعة حالة الطلب (Pending - Processing - Shipped - Delivered)

المطلوب تطبيقها OOP Concepts الـ

1. Encapsulation (التغليف)

- Private كل الكلاسات لازم تكون فيها بيانات
- Properties / Getters & Setters الوصول يتم عن طريق
- مثال:
 - Customer Name, Email
 - Product Price, Quantity

2. Abstraction (التجريد)

- اسمه مثلاً class abstract إنشاء:
 - PaymentMethod أو OrderAction
- implementation بدون methods يحتوي على
- الهدف إخفاء التفاصيل الداخلية

3. Inheritance (الوراثة)

- أساسي Class ترث من classes إنشاء

مثال:

- Product (Base Class)
 - ElectronicsProduct
 - ClothingProduct

4. Polymorphism (تعدد الأشكال)

- method overriding استخدام

مثال:

- CalculateDiscount() تختلف حسب نوع المنتج
- Payment Method أو طريقة الدفع تختلف حسب

5. Interfaces

- Interface زي:
 - IPayable
 - IShippable

مثال:

- Pay() لازم يطبق Payment Method كل
- Ship() لازم يطبق Order كل

Classes المطلوبة

1. Customer

- Id
- Name
- Email
- List of Orders

2. Product (Base Class)

- Id
- Name
- Price
- StockQuantity

3. Order

- Id
- Customer
- List of Products
- TotalPrice
- OrderStatus

4. OrderItem

- Product

- Quantity
- SubTotal

5. Payment (Abstract Class)

- Pay() method

6. Concrete Payment Methods

- CashPayment
- CreditCardPayment
- PaypalPayment

إضافية (اختياري) Features

- (VIP Customer) خصم حسب العميل
- حساب ضريبة
- تسجيل تاريخ الطلب
- إرسال إشعار عند تغيير الحالة

المطلوب تسليمه

- C# كامل بلغة Project (Console App أو Windows Forms)
- OOP استخدام كل مفاهيم
- كود منظم + أسماء واضحة
- كامل Order إمكانية تشغيل البرنامج وتجربة إنشاء

فكرة بسيطة للتطبيق

لما البرنامج يشتغل:

1. Customer المستخدم ينشئ
2. Products يضيف
3. Order ينشئ
4. يضيف منتجات للطلب
5. يختار طريقة الدفع
6. حالة الطلب + Total يعرض

الهدف من التمرين

- بشكل عملي مش نظري OOP فهم
- Inheritance أو Interface معرفة متى تستخدم
- بناء نظام قريب من الواقع

Implementation Options (Console OR Windows Forms)

Option 1: Console Application (Recommended for learning OOP)

Program Flow Example:

1. Create Customer
2. Add Products
3. Create Order
4. Add Products to Order
5. Choose Payment Method
6. Display Order Summary

Simple Console Structure:

```
static void Main(string[] args)
{
    Customer customer = new Customer("1", "Yahia", "yahia@email.com");

    Product p1 = new Product("1", "Laptop", 1000, 5);
    Product p2 = new Product("2", "Mouse", 50, 10);

    Order order = new Order("100", customer);

    order.AddProduct(p1, 1);
    order.AddProduct(p2, 2);

    order.SetPayment(new CashPayment());
    order.ProcessOrder();

    order.PrintSummary();
}
```

Option 2: Windows Forms Application (UI Version)

Forms Needed:

1. Customer Form

- TextBox: Name
- TextBox: Email
- Button: Save Customer

2. Product Form

- TextBox: Name
- TextBox: Price

- TextBox: Quantity
- Button: Add Product

3. Order Form (Main Form)

- ComboBox: Select Customer
- ListBox: Available Products
- NumericUpDown: Quantity
- Button: Add to Order
- Label: Total Price
- ComboBox: Payment Method
- Button: Confirm Order

Event Flow Example:

- btnAddProduct_Click → adds product to order list
- btnConfirmOrder_Click → calculates total + calls payment

Difference Between Console & Windows Forms

Console App	Windows Forms
Faster to build	Has UI
Good for OOP practice	Good for real applications
Text-based	Event-based system

★ Suggested for Students

- Start with Console first
- Then upgrade to Windows Forms

- Focus on OOP design before UI